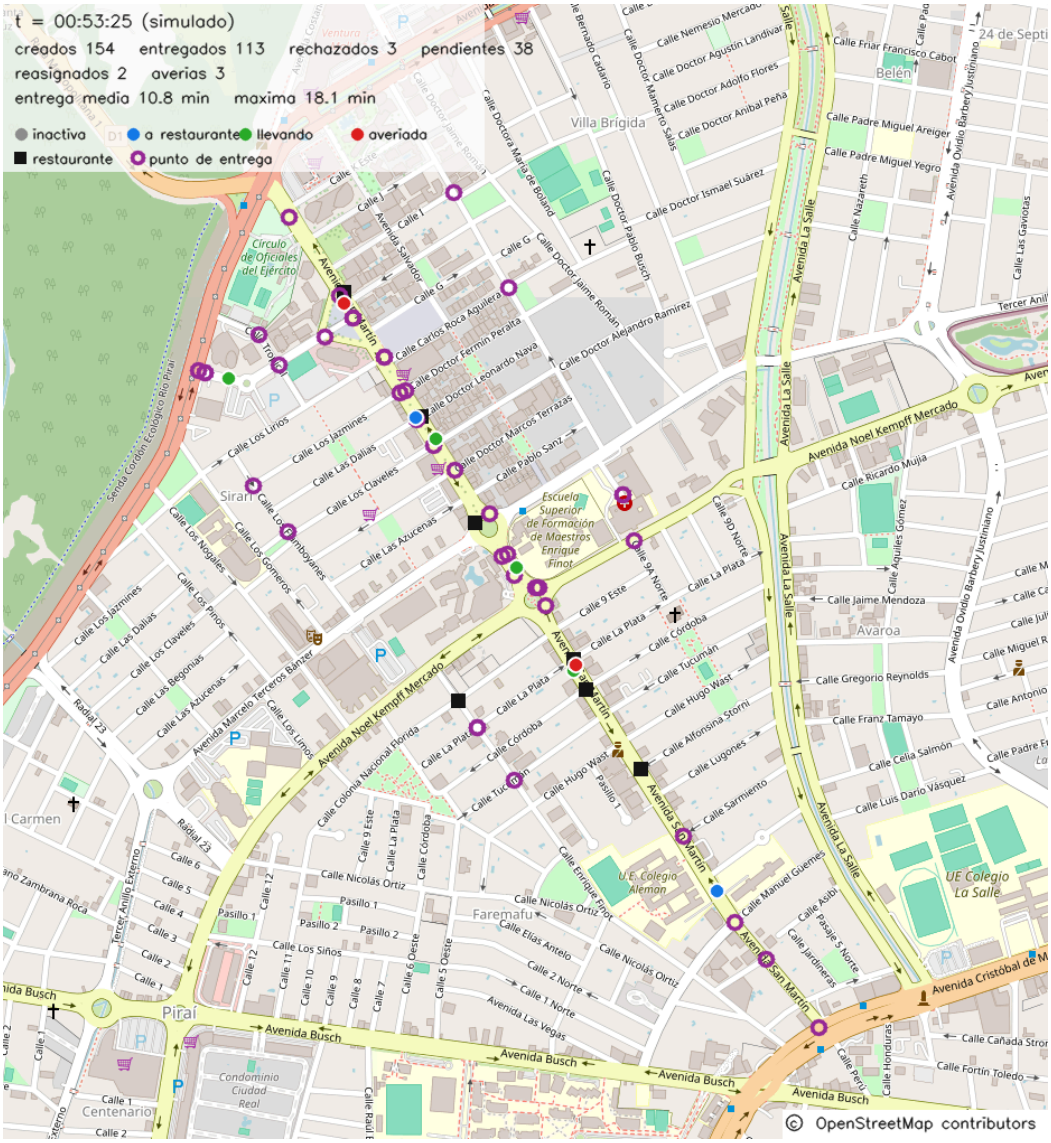
Equipetrol Delivery

Simulación concurrente de una empresa de delivery sobre un mapa real de Santa Cruz de la Sierra
Informe final del proyecto



La ventana durante una corrida: cada punto es una moto (gris inactiva, azul rumbo a un restaurante, verde llevando pedidos, rojo averiada); los cuadrados son restaurantes y los círculos, puntos de entrega. Mapa © OpenStreetMap contributors.

Materia	Computación: Concurrente, Paralela y Distribuida
Universidad	Universidad Católica Boliviana “San Pablo”, Santa Cruz
Autor	Erwin Alejandro Ojeda Justiniano
Repositorio	github.com/OjedaErwin123/Delivery_Equipetrol
Fecha	20 de octubre de 2026

1. Resumen

`delivery_sim` es un programa en **C++17** que simula una empresa de delivery en la Zona Equipetrol, entre el Segundo y el Cuarto Anillo. Los pedidos llegan en ráfagas. Un despachador los asigna a la moto que llegaría antes, calculando la estimación de todas las candidatas **al mismo tiempo**. Las motos circulan por las calles reales respetando el sentido único, hacen fila en los restaurantes y se averían; cuando una se avería, otra moto hereda sus pedidos. Todo se dibuja en vivo sobre la imagen de OpenStreetMap.

Cada actor es un hilo propio y se comunican solo por **monitores**: datos con su mutex, a los que se accede únicamente por métodos. Toda espera es **sin consumo de CPU** y se puede interrumpir para detener el programa. La concurrencia se hace solo con la biblioteca estándar; OpenCV se usa solo para la ventana y nlohmann/json solo para leer JSON.

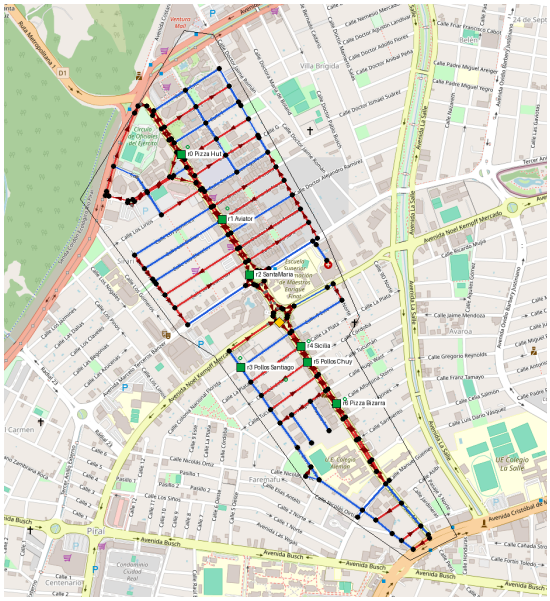
0	**32 / 32**	**2 %**	**≤ 3,4 s**	**0**
reportes de ThreadSanitizer	pruebas de `ctest`	CPU en reposo (sin ventana)	salida tras `SIGINT`, aun bajo Valgrind	errores de Memcheck sin ventana

Este informe resume el diseño y sus razones. El detalle completo, con enlaces a cada línea de código, está en el `README.md` del repositorio, que es el informe que pide el enunciado (DP.12).

2. El mundo simulado

Mapa y red vial

- Imagen de tiles estándar de OpenStreetMap (zoom 16, 933×1014 px), con sus límites geográficos exactos.
- La red vial sale de los datos de OpenStreetMap (`tools/osm_network.py`): **154 intersecciones, 219 calles** (151 de un solo sentido) y **7 restaurantes** reales distribuidos del 4.º al 2.º Anillo.
- El largo de cada calle se calcula con haversine. Los caminos mínimos se buscan con Dijkstra sobre el grafo dirigido, así que una moto nunca circula contra el sentido.
- Un pedido es `unreachable` si no hay camino dirigido desde su restaurante hasta el destino. Se calcula al arrancar, con un Dijkstra por restaurante en paralelo (`std::async`).



La red vial modelada sobre la imagen del mapa.

Tiempo

`t_sim = (ahora − inicio) × timeScale` con `steady_clock`, que nunca retrocede. Con `timeScale` = 60, una hora simulada dura un minuto real. Toda pausa representa tiempo simulado y es interrumpible: `wait_until` con predicado de parada.

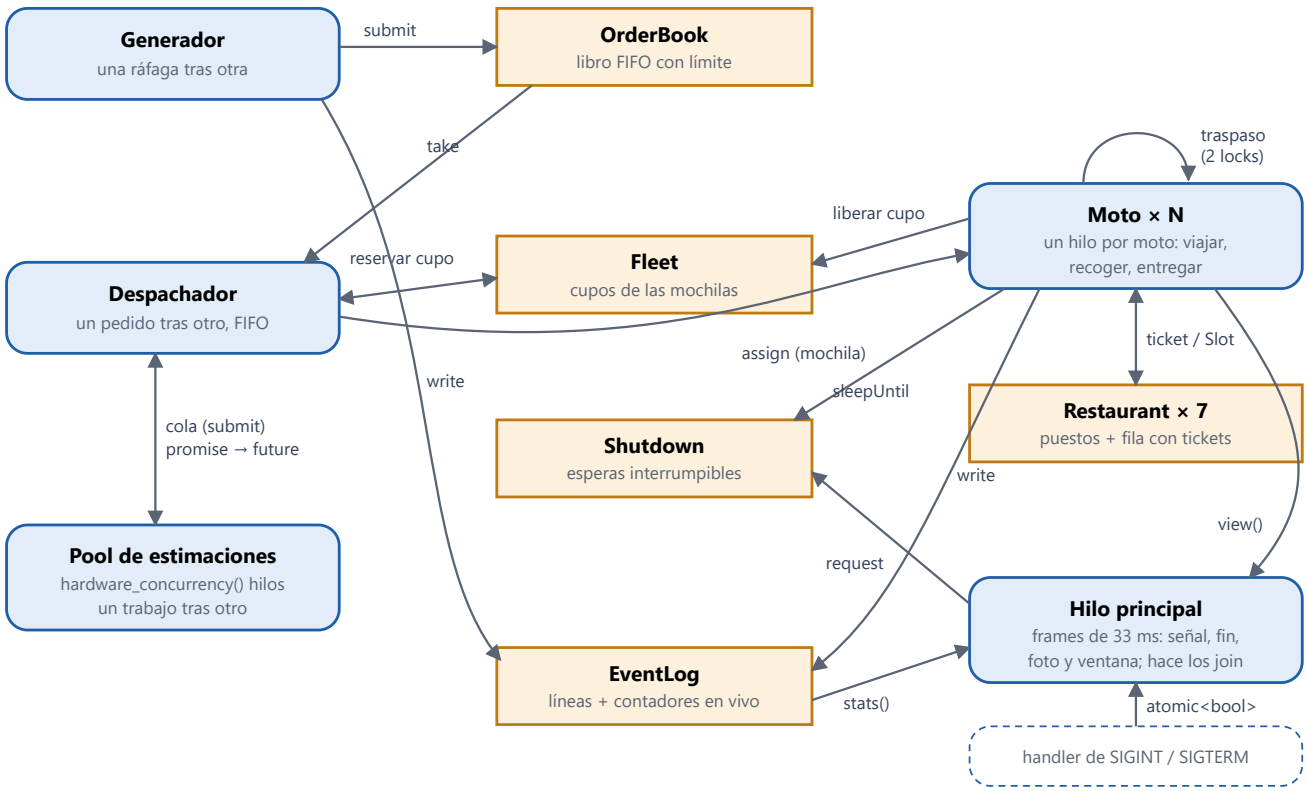
Vida de un pedido

1. **Creación:** el generador crea ráfagas de 1 a `burstMax` pedidos, reproducibles por semilla (`mt19937`). Se rechaza al instante si es `unreachable`, o si el libro ya tiene `maxPending` pedidos (`queueFull`).
2. **Libro FIFO:** el pedido espera su turno. El despachador lo toma y estima con las 8 motos con lugar más cercanas, todas a la vez. Si pasan `acceptTimeoutMs` sin poder asignarlo: `noCourier`.
3. **Asignación:** recién ahí empieza la preparación (`lista = asignado + prepTime`), mientras la moto viaja al restaurante.
4. **Restaurante:** la moto hace fila por orden de llegada para un puesto (`pickupSlots`), espera la comida y la entrega en el puesto dura 30 s simulados.
5. **Entrega:** la moto entrega los pedidos de su mochila (`bagCapacity`) en el orden en que se le asignaron.

Cada pedido termina en **exactamente un** estado final: `delivered`, `rejected` (`queueFull` | `unreachable` | `noCourier`) o `pending` si la simulación se detiene antes.

3. Arquitectura concurrente

Cada caja redondeada es un hilo; cada caja rectangular, un monitor; cada flecha, un canal por el que dos hilos se comunican o se esperan.



Hilos (azul) y monitores (naranja). El despachador también escribe en el log, y el generador y el despachador también esperan en `Shutdown`: esas flechas se omiten para que el diagrama se lea.

Hilo	Cuántos	Sobre qué itera	Cómo termina
Principal	1	Frames de 33 ms: fin por tiempo, bandera de la señal, foto para la ventana	Pide parar, aborta los monitores y hace `join` de todos
Generador	1	Llegadas de pedidos, durmiendo hasta cada una	Fin o detención; cierra el libro
Despachador	1	Pedidos del libro, de a uno	Libro cerrado y vacío, o detención
Pool de estimaciones	`hardware_concurrency()` (2 si da 0)	Trabajos de su cola	Su destructor descarta lo pendiente y hace `join`
Moto	`couriers`	El primer pedido de su mochila	Mochila cerrada y vacía, o detención
Alcanzabilidad	1 por restaurante	Un Dijkstra	Antes de empezar (`std::async` + `get`)

Ningún hilo se abandona: no hay `detach` en todo el código, y todos se unen antes de imprimir el resumen.

4. Las decisiones de concurrencia más importantes

4.1 Despacho: estimaciones simultáneas que nunca frenan al sistema

Para cada pedido, el despachador encola una estimación por candidata en un **pool de hilos propio**: una cola monitor con hilos fijos. Cada trabajo devuelve su resultado por un `std::promise`, y el despachador espera los futures con `wait_until` hasta `quoteTimeoutMs` en tiempo real. Una estimación que falla, por ejemplo una moto sin camino, lanza una excepción. La excepción cruza de hilo con `set_exception`, sale en `get()`, se reporta en stderr y ese candidato se descarta.

> **Por qué no `std::async`:** su future **bloquea al destruirse** hasta que la tarea termina. Si el despachador dejara de esperar una estimación tardía, el destructor lo congelaría igual. El future de un `promise` no bloquea: la tarea tardía termina en el pool y su resultado se descarta. Además, con 64 motos hay 64 *tareas* por pedido, no 64 hilos.

Cada estimación trabaja sobre una **copia** del plan de la moto, así que corren en paralelo sin locks. Entre estimar y asignar el estado puede cambiar (check-then-act). Por eso la asignación **revalida** con el lock de la moto: si ya no tiene lugar o se averió, se prueba la siguiente mejor.

4.2 Restaurantes: puestos y orden de llegada

Un `condition_variable` solo no garantiza el orden de llegada, porque cualquiera puede ganar el lock. Por eso cada moto saca un **ticket** al llegar, con el lock tomado, y entra solo el ticket siguiente, y solo si hay un puesto libre. El puesto es un objeto RAII move-only (`Slot`) que se libera solo al destruirse, aunque la moto abandone por la detención.

El log no tenía cómo mostrar el orden de llegada: ningún evento del enunciado dice cuándo llega una moto. Por eso `pickupStarted` lleva la propiedad `arrivedAt`, leída con el **mismo lock** que el ticket. Si se leyera antes, el log podría mostrar un desorden que no ocurrió. `tools/verificar_orden_llegada.py` lo comprueba sobre cualquier log.

4.3 Averías: traspaso de todo o nada, sin deadlock

Al averiarse, la moto traspasa **todos** sus pedidos sin entregar a un solo receptor: el que llega antes por calle. Mover las dos mochilas necesita **dos locks a la vez**. Si A le traspasa a B mientras B le traspasa a A, cada hilo pide los mismos locks al revés, que es la receta del deadlock. `std::scoped_lock` los toma con el algoritmo de `std::lock`, que lo evita. El resto del programa sigue un orden global: moto → flota → `Shutdown` → log. Los pedidos ya recogidos se van a buscar al punto de la avería.

4.4 Esperar sin consumir CPU, y poder detenerse en menos de 5 s

- Cada espera es un `wait` con predicado, robusto a despertares espurios. Un despertar sin novedades vuelve a dormir.
- Las pausas de tiempo simulado son `wait_until` sobre el monitor `Shutdown`: una orden de parar las despierta a todas a la vez.
- El handler de la señal **solo** pone un `std::atomic<bool>` lock-free, porque tomar un lock o notificar dentro de un handler no es seguro. La lee el bucle de frames, que ya se despierta cada 33 ms para dibujar.
- Cada monitor tiene su `abort()/close()` con `notify_all`, así nadie queda dormido al detener.

4.5 El log dice la verdad

Cada cambio de estado y su línea en el log se hacen **en la misma sección crítica**. Así un pedido nunca aparece entregado y todavía en una mochila. El `EventLog` escribe cada línea completa con su lock, y actualiza los contadores del panel en esa misma sección: la ventana nunca contradice el log. Tres atomics separados no darían esa foto consistente.

5. Mapa de conceptos

Versión resumida. El README tiene la de la sección 10, con el enlace a cada línea.

Carp.	Concepto	Dónde y por qué
01	Hilos explícitos con destino deliberado	Motos, despachador y generador son `std::thread`, todos unidos antes del resumen; el pool une los suyos en su destructor. Un hilo suelto podría escribir en el log después del resumen.
01	Decisión según el hardware	El pool tiene `hardware_concurrency()` hilos (2 si da 0): más hilos que núcleos solo agregan cambios de contexto (experimento 2).
02	Por valor, referencia y movimiento	La estimación copia el plan (puede terminar tarde); el traspaso captura por referencia objetos que viven hasta el `join`; el pedido entra a la mochila con `std::move`.
02	Pedidos por movimiento, en un solo lugar	`Order` es move-only: pasa del generador al libro, al despachador, a la mochila y, en un traspaso, a otra mochila. Copiable, podría quedar en dos mochilas.
03	Métodos de objetos compartidos en hilos; contenedores de hilos	`Courier::run` en su hilo, guardado en un `vector<thread>` con `emplace_back`.
03	Propiedad compartida; sin punteros crudos dueños	Motos en `shared_ptr`, mapa en `shared_ptr<const World>`, pool en `unique_ptr`. Los punteros crudos que quedan no son dueños.
04	Future con política explícita	`std::async(std::launch::async)` para la alcanzabilidad, un Dijkstra por restaurante. Sin la política podrían diferirse y correr uno tras otro.
04	Promise/future de un solo uso	Un `promise` por trabajo del pool. Su future no bloquea al destruirse (sección 4.1).
04	Espera con límite; excepción entre hilos	`wait_until` con `quoteTimeoutMs`; `EstimateError` por `set_exception` y `get()`.
05	Argumento de “sin data race” por objeto	Tabla objeto por objeto en el README: cada objeto compartido es un monitor, es inmutable después de lanzar los hilos, o es un atomic.
05	Tipo seguro de mover	`Restaurant::Slot`: libera en el destructor, copia borrada y el movimiento deja el origen vacío. `Trip` sigue la regla de cero.
06	Locks RAII; secciones críticas cortas	Solo `lock_guard`, `unique_lock` y `scoped_lock`. Se copia bajo el lock y se calcula afuera.
06	Dos locks a la vez sin deadlock	El traspaso por avería con `std::scoped_lock` (sección 4.3).
07	Monitores	`OrderBook`, `Fleet`, `Restaurant`, `Shutdown`, `EventLog`, la cola del pool y la mochila de cada moto.
07	Esperas con predicado y despertar final	Moto sin pedidos, fila por ticket, despachador esperando cupo, hilos del pool y tiempo simulado: todas con predicado, y `abort()` + `notify_all` para el apagado.

6. Cómo se verificó

- **32 pruebas en `ctest`**, una por pieza. Por ejemplo:
  - `transfer_check`: dos hilos traspasan A→B y B→A 2000 veces a la vez, sin deadlock ni pedidos perdidos o duplicados;

    - `restaurant_check`: con el FIFO roto a propósito, falla en 20 de 20 corridas;
    - `signal_check`: señales en momentos al azar.

- En cada PR, la misma batería: build con g++ 15.2, clang 21.1 y CMake 3.11.4 real (el mínimo del enunciado), con 0 warnings; las pruebas bajo ThreadSanitizer, varias veces cada una; Memcheck sobre las pruebas; y una corrida completa de `delivery_sim`, con su log verificado contra las reglas de las secciones 4, 6 y 7.
- CPU en reposo, con 8 motos y 0 pedidos durante 20 s reales: **2 %** sin ventana y 15 % con ventana, que es el costo de dibujar 30 cuadros por segundo.
- Salida tras `SIGINT`/`SIGTERM` con código 0: 2,0 s en el build normal, 3,1 s bajo TSan y 3,4 s bajo Valgrind. Esos tiempos incluyen los 2 s en que la ventana muestra el resumen.

7. Experimentos (DP.10)

Programa aparte (`delivery_experiments`), sin ventana, con 1 hora simulada por corrida. En cada experimento se cambia una sola cosa. Los datos en bruto de cada corrida (configuración, log y resumen) están en `experiments/`.

7.1 Candidatos: todas las motos contra las k más cercanas → cambio de diseño

64 motos, un pedido cada 1,5 s simulados, semillas 1–3. El tiempo de ronda es tiempo real: desde que se encolan las estimaciones de un pedido hasta tener los resultados.

Candidatos	Build	Ronda media (ms)	Entregados	`queueFull`
todas	normal (-O2)	0,815	812,7	1375
k = 8	normal	0,572	798,0	1389
todas	ThreadSanitizer	8,686	**579,3**	1697
k = 8	ThreadSanitizer	2,043	**782,7**	1406

Con el build normal casi no hay diferencia. Cuando estimar es caro (bajo TSan, como lo corren los evaluadores, o en mapas de cientos de intersecciones), la ronda con todas las motos tarda más que lo que tarda en llegar el próximo pedido: el despachador se vuelve el cuello de botella y se entrega un 26 % menos. **Cambio de diseño:** estimar con las 8 motos con lugar más cercanas, y con todas solo si ninguna de esas 8 da una estimación usable. Antes y después, bajo TSan: **579 → 783 entregados**.

7.2 Tamaño del pool

Hilos	Ronda media (ms)	Speedup	Eficiencia
1	1,753	1,00	1,00
2	1,234	1,42	0,71
4	0,906	1,93	0,48
8	0,778	2,25	0,28

El speedup se ajusta a la **ley de Amdahl** con ~36 % de parte secuencial: el despachador encola, espera y ordena en un solo hilo. Con 4 hilos Amdahl predice 1,93 y se midió 1,93; con 8 predice 2,27 y se midió 2,25. En 4 núcleos, pasar de 4 a 8 hilos gana solo un 14 %, lo que confirma usar `hardware_concurrency()` hilos.

7.3 Averías y capacidad de la mochila

8 motos, semillas 1–5; promedios.

breakdownProbability	bagCapacity	Entregados	noCourier	Reasignados	Espera al traspaso (mediana / p90)
0	3	118,0	16,0	0,0	—
0,02	3	93,2	36,4	22,4	103 s / 426 s
0,1	3	67,0	62,0	49,6	50 s / 300 s
0,02	1	68,6	70,4	7,8	35 s / 119 s
0,02	5	107,2	16,6	28,2	1 s / 283 s

Lo que cuesta caro de una avería no es la reparación sino el **traspaso**: hace falta una moto con lugar para *toda* la mochila, y con mochila 3 casi todas están llenas. Con mochila 5 casi siempre hay lugar enseguida. Se probaron y **descartaron** dos variantes, con sus números: sin prioridad del traspaso se entregan 78,6, y con un receptor designado, 88,0. Las dos quedan por debajo del diseño elegido, con 93,2.

8. Análisis con herramientas (DP.11)

Corrida completa de `config/equipetrol.json` **con la ventana abierta**: ráfagas, averías y traspasos, detenida con `kill -INT` a los 40 s reales. La salida cruda, sin editar, está en `docs/herramientas/`, junto con las versiones y los comandos exactos.

Herramienta	Comando	Resultado
ThreadSanitizer (g++ 15.2)	`cmake -DENABLE_TSAN=ON .. && make`	**0 reportes.** Sin las supresiones, 5 reportes, todos entre hilos de Qt y libdbus; ninguno en nuestro código
Valgrind 3.26 Memcheck	`valgrind --leak-check=full --track-origins=yes ./delivery_sim …`	**Sin ventana: 0 errores, 0 bytes perdidos.** Con ventana: 4 errores, todos de bibliotecas (abajo)

Hallazgos y cambios

1. **OpenCV paraleliza por dentro con TBB**, que está prohibido. `cv::setNumThreads(0)` hace que dibuje en el hilo que lo llama, sin crear hilos.
2. **Qt tiene carreras propias** en su hilo de DBus. Se suprimen **solo** los reportes cuya pila pasa por Qt, DBus o GTK, y solo en el build de TSan. Un programa mínimo con una carrera inyectada a propósito demuestra que las carreras nuestras se siguen reportando.
3. **Un defecto nuestro, que encontró la verificación bajo TSan:** un check-then-act entre el despachador y una avería. El despachador reservaba el cupo de una moto, la moto se averiaba y traspasaba todo, y aun así recibía el pedido. Ahora la moto revalida con su lock al recibir una asignación. Hay una prueba determinística del caso.
4. **Memcheck con ventana: de 25 errores a 4.**
   - 18 eran falsos positivos del JIT de expresiones regulares de Qt: se desactivó, porque la ventana no lo usa.
   - 3 eran fugas de OpenGL/Mesa, que la ventana tampoco necesita: se desactivó.
   - 1 aparecía con la pila de un hilo nuestro, pero era de Qt: se reordenó la apertura de la ventana.

Lo que queda, y por qué no se puede corregir desde nuestro código

Error	Dónde	Explicación
1 “Invalid read” al cerrar la ventana	OpenCV 4.10 (`window_QT.cpp`)	Error de OpenCV: al destruir la última ventana, borra su objeto `GuiReceiver` mientras todavía ejecuta un método suyo. Sigue en la versión actual. No destruir la ventana deja 2,8 MB perdidos.
256 B “definitely lost”	fontconfig	Una reserva única al leer la configuración de fuentes.
160 B y ~620 B “possibly lost”	Qt (DBus, X11)	Dos hilos internos de Qt que Qt nunca une al salir.

9. Decisiones propias y limitaciones

Donde el enunciado no fija algo, se decidió y se documentó en el README. Por ejemplo: el destino de un pedido es una intersección al azar sin restaurante; `bagCapacity` cuenta los pedidos asignados y una visita al restaurante recoge todos los de ese restaurante; un “viaje”, para la probabilidad de avería, es cada tramo; y cerrar la ventana, `q`/`Esc` o el botón Terminar detienen la simulación igual que una señal.

Limitaciones conocidas: la estimación no cuenta la fila del restaurante, porque depende de quién llegue antes; el destino es siempre una intersección, no un punto a mitad de cuadra; y los 4 errores de Memcheck con ventana dependen de las bibliotecas del sistema, así que con otro backend de OpenCV (por ejemplo GTK) pueden ser otros.

10. Uso de inteligencia artificial

En el desarrollo de este proyecto **se usó inteligencia artificial**: Claude, de Anthropic, a través de Claude Code, como asistente de programación. Se usó para:

- discutir el diseño y las alternativas de cada componente, a la luz de los ejemplos del curso;
- escribir buena parte del código, las pruebas y los scripts de `tools/`;
- correr la verificación de cada cambio (builds, ThreadSanitizer, Valgrind, mediciones) y los experimentos;
- redactar la documentación: el README, las descripciones de los PRs y este informe.

Las decisiones las tomó el autor. Las ambigüedades del enunciado se le presentaron como opciones con su justificación y él eligió. Cada cambio entró por un PR revisado por el autor antes del merge. Los commits hechos con asistencia llevan la línea `Co-Authored-By: Claude`, así que el historial del repositorio muestra qué partes se hicieron así.

11. Compilar y ejecutar

Dependencias: CMake ≥ 3.11, g++ ≥ 7 o clang, OpenCV ≥ 4.1 (`libopencv-dev`), y Python 3 solo para los scripts de `tools/`. Datos del mapa © OpenStreetMap contributors, licencia ODbL.

```
mkdir build && cd build && cmake .. && make        # la primera vez descarga nlohmann/json
./delivery_sim ../config/equipetrol.json              # ventana; el log queda en events.log
python3 ../tools/verificar_orden_llegada.py events.log # orden de llegada en los restaurantes
ctest                                                 # las 32 pruebas
mkdir ../build-tsan && cd ../build-tsan && cmake -DENABLE_TSAN=ON .. && make
```